

CHUYÊN ĐỀ 10

Xử lý chuỗi với thuật toán KMP và cấu trúc dữ liệu Trie

Interactive Text Search & Autocomplete System

Nhóm chuyên đề 10 · Trình bày: Huỳnh Phát Lợi · Võ Phú Vinh · Thanh Tú

Môn: Các thuật toán tối ưu · Tháng 8 năm 2026

Thành viên và phần đóng góp

Huỳnh Phát Lợi

Trưởng nhóm • KMP • tích hợp

Thiết kế kiến trúc; build_ips, kmp_search; Aho–Corasick; CLI/Web demo

Võ Phú Vinh

Lý thuyết • báo cáo

Định nghĩa, chứng minh tính đúng, phân tích khấu hao; biên tập nội dung trình bày

Thanh Tú

Trie • kiểm thử • thực nghiệm

Trie/autocomplete; baseline; test biên/đối kháng/ngẫu nhiên; benchmark và biểu đồ

Phân công là trách nhiệm chính; cả ba thành viên cùng review phần lỗi và có thể giải thích KMP, Trie, baseline và kết quả thực nghiệm.

Hai bài toán nghe giống nhau, cấu trúc khác hẳn nhau

	Tìm mẫu trong văn bản	Gợi ý theo tiền tố
Cái gì biết trước?	không gì cả	toàn bộ từ điển
Tiền xử lý phía nào?	MẪU	DỮ LIỆU
Thuật toán	KMP	Trie
Phụ thuộc $ \Sigma $?	KHÔNG	CÓ

Cái gì được biết trước quyết định nên tiền xử lý phía nào.

Đó là lý do đề bài yêu cầu hai module độc lập — trong mã nguồn, src/kmp và src/trie không import lẫn nhau.

Phát biểu bài toán

MODULE A — KMP

Vào: văn bản T ($|T| = n$), mẫu P ($|P| = m \geq 1$)

Ra: MỌI vị trí xuất hiện, kể cả overlapping

Thời gian: $O(n + m)$, số so sánh $\leq 2n$

Bộ nhớ phụ: $O(m)$ cho mảng LPS

T	P	Kết quả
aaaa	aa	[0, 1, 2]
mississippi	issi	[1, 4]
abc	abcd	[] ($m > n$)

MODULE B — TRIE

Bốn thao tác bắt buộc:

insert · exact search · prefix query

autocomplete(q, k) có giới hạn kết quả

Σ' khai báo rõ: 38 ký tự (mặc định), 105 (tiếng Việt)

Truy vấn	Kết quả
search("ti")	False ← chỉ là tiền tố
starts_with("ti")	True ← ngược lại
autocomplete("t", 2)	[tin, tinh], truncated

Prefix function π — trái tim của KMP

$\pi[i]$ = độ dài border dài nhất của $P[0..i]$

border = chuỗi vừa là tiền tố thật sự, vừa là hậu tố thật sự

i	0	1	2	3	4	5	6
P[i]	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

$\pi[4] = 3$ vì "ababa" có border dài nhất là "aba"

Bổ đề (chuỗi border)

Tập MỌI border của S = $\pi[k-1] \rightarrow \pi[\pi[k-1]-1] \rightarrow \dots \rightarrow 0$

Khi một border độ dài L không dùng được, border tiếp theo dài đúng $\pi[L-1]$ — ta nhảy thẳng, không thử tuần tự.

Vì sao con trỏ văn bản không bao giờ lùi

Bất biến (BB-A): trước khi xử lý $T[j]$, i là độ dài lớn nhất $< m$ sao cho $P[0..i-1]$ là hậu tố của $T[0..j-1]$

Trạng thái là MỘT SỐ NGUYÊN duy nhất — không phải một tập vị trí. Đó chính là lý do j không cần lùi.

```
for j <- 0 to n-1:
  while T[j] != P[i] and i > 0:
    i <- pi[i-1]          # dịch MẪU, j KHÔNG lùi
  if T[j] = P[i]: i <- i+1
  if i = m:
    báo khớp tại j-m+1
    i <- pi[m-1]          # KHÔNG phải i <- 0 !
```

Dòng quan trọng nhất: $i \leftarrow \pi[m-1]$

$T = \text{"aaaa"}, \quad P = \text{"aa"}, \quad \pi = [0, 1]$

j	i sau khi xử lý	Hành động
1	$2 = m$	báo vị trí 0, đặt $i \leftarrow \pi[1] = 1$
2	$2 = m$	báo vị trí 1, đặt $i \leftarrow 1$
3	$2 = m$	báo vị trí 2

$i \leftarrow \pi[m-1]$

Kết quả [0, 1, 2] ✓ đủ nghiệm

Giữ lại hậu tố dài nhất còn là tiền tố của mẫu

$i \leftarrow 0$

Kết quả [0, 2] ✗ MẤT một nghiệm

Vứt bỏ toàn bộ thông tin đã đọc, phải bắt đầu lại

Chặn $O(n+m)$: lập luận khấu hao

Định lý. Số phép so sánh $\leq 2n$ ở vòng chính, $\leq 2m$ khi xây π .

Chứng minh bằng hàm thế $\Phi = 2j - i$:

- 1 So sánh THÀNH CÔNG**
 i tăng 1, rồi j tăng $\rightarrow$ không quá n lần
- 2 So sánh THẤT BẠI ($i > 0$)**
 i GIẢM NGẶT vì $\pi[i-1] < i$ (border là thật sự)
- 3 Cân bằng tăng/giảm**
 i chỉ tăng 1 mỗi lần khớp $\Rightarrow$ tổng tăng $\leq n \Rightarrow$ tổng giảm $\leq n$

$\leq 2n$

$|\Sigma|$ KHÔNG xuất hiện

Bộ nhớ phụ $O(m)$

Bốn bất biến của Trie

I1 Mỗi nút $\leftrightarrow$ ĐÚNG MỘT tiền tố

I2 $\text{is_end} \iff$ tiền tố đó là một từ trong D

I3 $\text{word_count} =$ số lần từ đó đã insert

I4 $\text{prefix_count} =$ tổng word_count của CẢ CÂY CON

I1 $\Rightarrow$ chia sẻ tiền tố

Bộ nhớ tỉ lệ số tiền tố PHÂN BIỆT, không phải tổng độ dài các từ. Đo được 0,31–0,82 nút mỗi ký tự.

I4 $\Rightarrow$ đếm trong $O(|q|)$

“Có bao nhiêu từ mang tiền tố q?” không cần duyệt cây con. Demo hiển thị “8 / 2 513 kết quả” nhờ đúng bất biến này.

autocomplete: DFS dừng sớm

```
path <- list(q); stack <- [FRAME(node_q)]
while stack khác rỗng:
    u <- frame hiện tại
    if u.is_end:
        if |out| = k:
            truncated <- true; BREAK    # dừng sớm
        out.append(join(path))
    lấy cạnh tiếp theo theo thứ tự TĂNG; push FRAME(con)
```

Thứ tự XÁC ĐỊNH

thứ tự từ điển $\Rightarrow$ đối chiếu được với baseline

Stack tường minh

không đệ quy $\Rightarrow$ chạy được với từ 50 000 ký tự

Cờ truncated chính xác

false $\Rightarrow$ đã trả HẾT kết quả

Đo thực tế: cây con có 2 513 từ, $k = 1 \Rightarrow$ duyệt 6 nút (0,08 % cây con)

Số nút duyệt $\approx 3k$ — tỉ lệ với k , KHÔNG tỉ lệ với kích thước cây con. Đúng thứ ô tìm kiếm cần: người dùng chỉ xem 8–10 gợi ý.

DEMO

Hai module cùng chạy trên một luồng người dùng thật

1 Gõ tiền tố

Trie gợi ý, hiện số nút đã duyệt

2 Chọn gợi ý

KMP tìm, tô sáng, hiện bảng π

3 Nhiều mẫu

Aho–Corasick, so với KMP lặp

276 test · 3 baseline độc lập

276

test tự động + 10 doctest

18

test ĐỐI KHÁNG

22

test ngẫu nhiên đối chiếu

6

property test

Ba baseline, **KHÔNG** dùng lại một dòng nào của src/

Baseline	Đối chiếu cho	Vì sao độc lập
naive_search $O(n \cdot m)$	KMP	thuật toán khác hẳn, ngây thơ tới mức đọc là tin
str.find của CPython	KMP	viết bằng C, dùng thuật toán two-way
naive_build_lps $O(m^3)$	build_lps	dịch TRỰC TIẾP từ định nghĩa border
linear_autocomplete_sorted	Trie	quét list đã sắp; không set/sort trong vùng đo

Test đối kháng: mỗi ca có mục tiêu tấn công

Trường hợp	Tấn công vào	Kết quả
$T = a^n, P = a^{m-1}b$	ca xấu nhất của naive	$KMP \leq 2n$, naive $\sim n \cdot m$
chuỗi FIBONACCI	vòng while lùi π (dãy border dài nhất)	vẫn $\leq 2n$
$P = \text{"abab"}$ tuần hoàn	bảo vệ tính chất overlapping	999 nghiệm, đủ
cây chối: 2 000 từ, tiền tố 100 ký tự	cơ chế early stop của Trie	duyet < 300 nút
từ dài 50 000 ký tự	giới hạn đệ quy của Python	pass (stack tường minh)
ký tự lạ ở CUỐI từ dài	cây bị bắn khi insert thất bại	num_nodes không đổi

Chuỗi Fibonacci có dãy border dài nhất trong các chuỗi cùng độ dài — nếu lập luận khẩu hao sai ở đâu đó, chính ca này sẽ làm vượt chặn $2n$.

Chặn $2n$ chặt tới bốn chữ số thập phân

Văn bản	n	KMP cmp/n	naive cmp/n	Tăng tốc
natural	10^6	1,143	—	—
random_s26	10^6	1,039	—	—
repetitive	10^5	1,000	32,5	32,5×
uniform	10^5	1,9994	64,0	32,0×
uniform	10^6	1,9999	—	—

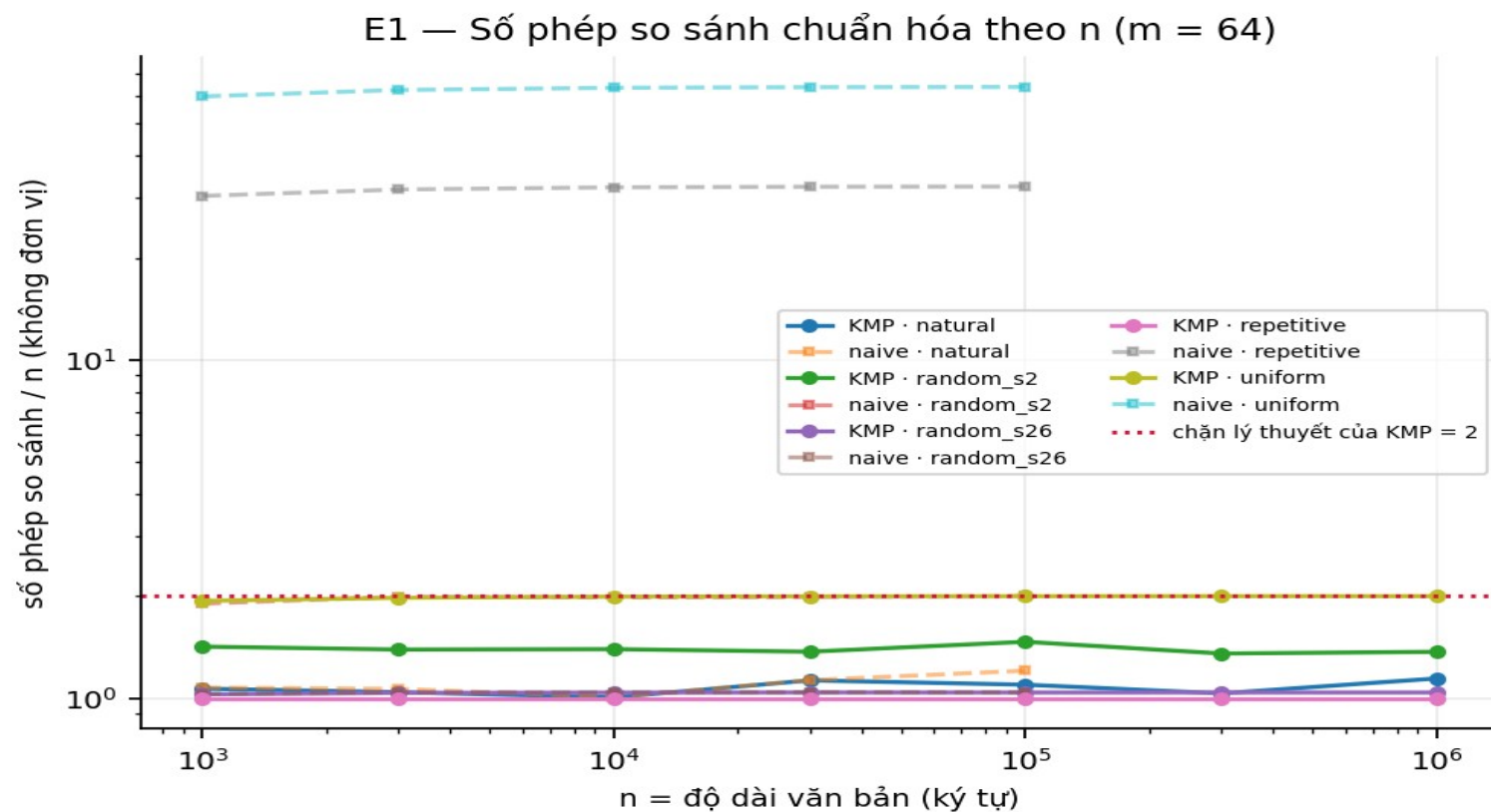
1,9999 · n

sát cận $2n$, chưa bao giờ vượt

Trung thực:

Trên dữ liệu natural-like tổng hợp theo Zipf, KMP $\approx$ naive (1,0–1,1×). Với $|\Sigma|$ lớn, naive đã gần tuyến tính. Lợi thế KMP chỉ hiện khi dữ liệu lặp.

Số phép so sánh chuẩn hóa theo n



KMP: tỉ số PHẪNG và luôn dưới 2 $\Rightarrow$ tuyến tính, khớp lập luận khẩu hao. naive: tỉ số TĂNG theo n trên văn bản đối kháng. Nguồn: e1_scaling_n.csv

m không ảnh hưởng KMP • N không ảnh hưởng Trie

E2 — m biến thiên 1 024 lần (n = 30 000)

m	KMP cmp/n	naive cmp/n	LPS cmp/m
4	1,9999	4,0	1,25
64	1,9979	63,9	1,95
1 024	1,9659	—	1,997

Chặn 2m của BUILD_LPS cũng chặt: 1,997

Cột KMP giữ nguyên khi m biến thiên 1 024 lần $\Rightarrow$ chặn là 2n, không phải hàm của m.

E3 — N biến thiên 500 lần

N	Trie	Tuyến tính	Tăng tốc	search
1 000	14,4 μ s	19,3 μ s	1,3×	6 nút
100 000	7,1 μ s	1 832 μ s	258,6×	10 nút
500 000	8,6 μ s	9 161 μ s	1 062×	10 nút

search duyệt 6–10 nút = ĐỘ DÀI TỪ, độc lập N

Trie gần như phẳng; baseline quét tăng theo N và không gồm chi phí set/sort.

Số đếm nút là bằng chứng sạch hơn cả thời gian — chính xác tuyệt đối, không bị nhiễu bởi ngôn ngữ hay phần cứng.

Mở rộng: Aho–Corasick nối hai module

KMP (module A)	Aho–Corasick
π trên MỘT mẫu, lưu bằng mảng	failure link trên TRIE nhiều mẫu
$\pi[i]$ = border dài nhất của $P[0..i]$	$fail(u)$ = hậu tố dài nhất còn là tiền tố của một mẫu
lùi theo dãy border	lùi theo dãy failure link
$O(n + m)$	$O(n + \sum P_i + \text{số khớp})$

Aho–Corasick = cấu trúc của module B + ý tưởng của module A

Số mẫu	AC (so sánh)	KMP lặp (so sánh)	Tỉ số
1	109 539	108 737	0,99× (chậm hơn!)
10	144 341	1 057 779	7,3×
100	148 241	10 440 185	70,4×

Giới hạn — nói thẳng

Nhiều truy vấn / cùng văn bản

KMP xử lý MẪU $\Rightarrow$ mỗi truy vấn vẫn $O(n)$

$\rightarrow$ suffix automaton

Tìm mờ, lỗi chính tả

Trie chỉ khớp tiền tố CHÍNH XÁC

$\rightarrow$ Trie + Levenshtein

Xếp hạng theo tần suất

phải xem hết cây con $\Rightarrow$ MẤT early stop

$\rightarrow$ lưu sẵn top-k mỗi nút

KHÔNG khẳng định KMP nhanh hơn `str.find` — hàm đó viết bằng C, nhanh hơn hàng chục lần.

Giá trị của module A: mọi vị trí kể cả overlapping, với chặn $2n$ CHỨNG MINH ĐƯỢC — và là nền cho Aho–Corasick.

8 / 9

khẳng định lý thuyết được số liệu xác nhận đầy đủ.

Khẳng định thứ 9 (mảng tốn bộ nhớ hơn hash map) đúng HƯỚNG nhưng chỉ $1,50\text{--}1,53\times$ ở $|\Sigma'| = 26$ — do chi tiết cài đặt CPython. Em giữ nguyên kết quả này kèm phân tích thay vì lược bỏ.

Xin cảm ơn thầy và các bạn đã lắng nghe.